Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

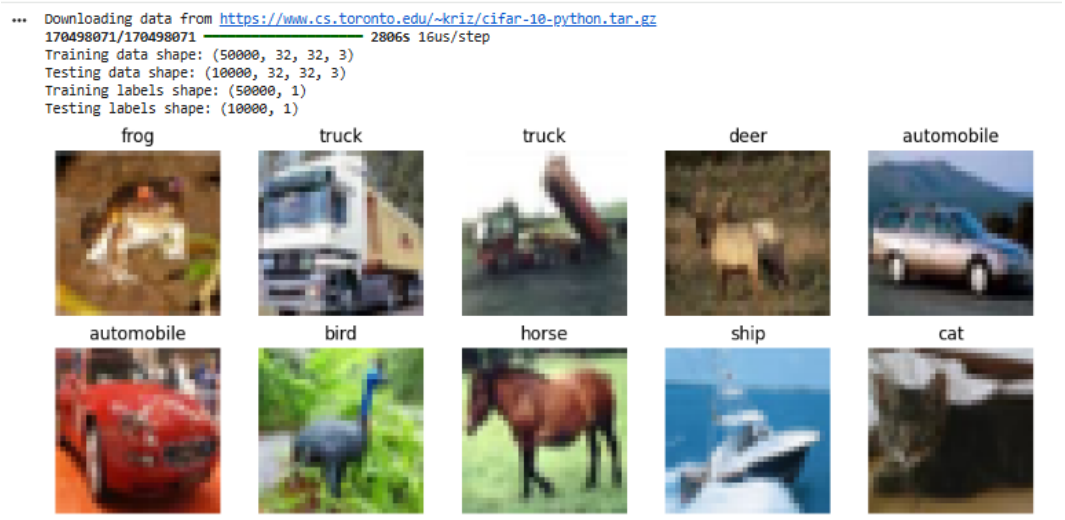


Figure 1: Sample images from the CIFAR-10 dataset.

Inference: The images show different CIFAR-10 classes, such as frog, truck, deer, automobile, bird, horse, ship, and cat. Despite the low resolution, the objects remain recognizable from their visual features.

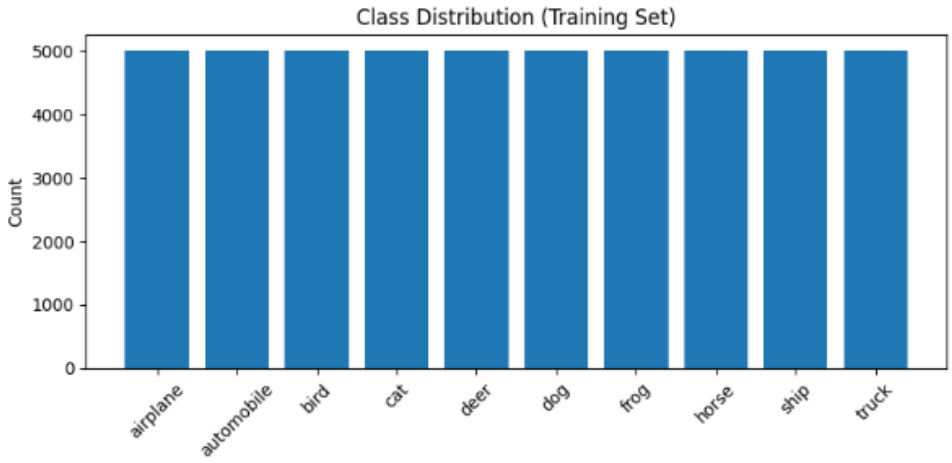


Figure 2: Distribution of classes in the training set.

Inference: The class distribution is exactly equal across all 10 CIFAR-10 classes, with 5,000 images per class, confirming the dataset is perfectly balanced.

Task 2
Implement a convolution layer.
Compare
- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

```
Kernel size 3x3 -> Feature map shape: (1, 30, 30, 8)
Kernel size 5x5 -> Feature map shape: (1, 28, 28, 8)
Kernel size 7x7 -> Feature map shape: (1, 26, 26, 8)
```

Figure 3: Shapes of the feature maps.

Inference: Larger kernels produce smaller feature maps because they cover more pixels at a time.

Task 3
Study hyperparameters.
Compare
- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

```
Stride=1, Padding=same -> Output shape: (1, 32, 32, 8)
Stride=1, Padding=valid -> Output shape: (1, 30, 30, 8)
Stride=2, Padding=same -> Output shape: (1, 16, 16, 8)
Stride=2, Padding=valid -> Output shape: (1, 15, 15, 8)
```

Figure 4: Hyperparameter comparison.

Inference: With stride = 1, the output feature map is larger, while stride = 2 reduces it significantly. Same padding preserves more spatial size than valid padding, especially when the stride is 2.

Task 4

Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

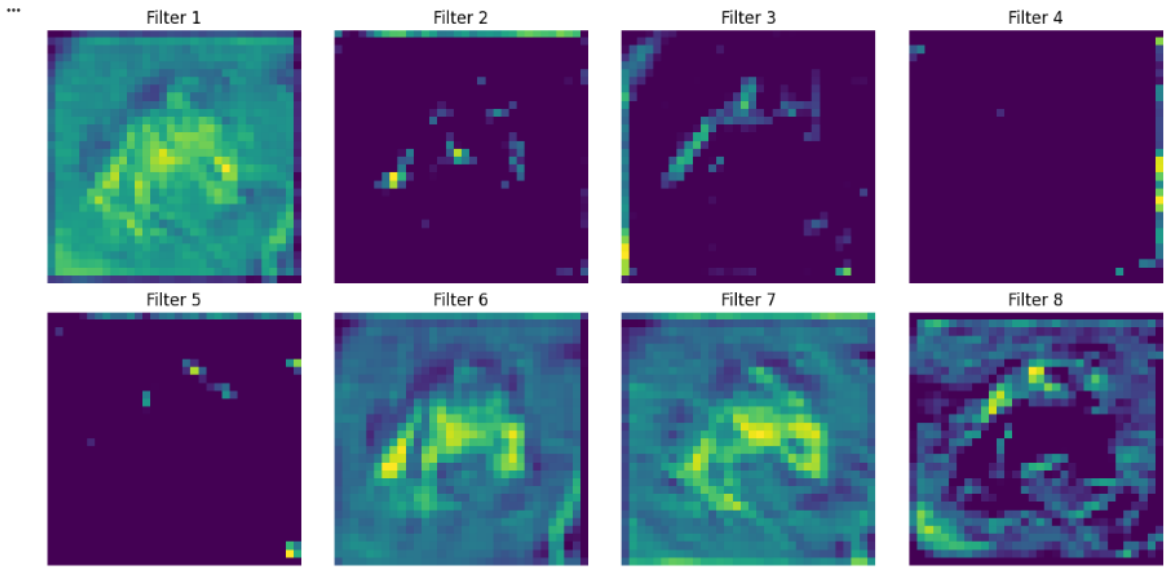


Figure 5: Feature maps from the first convolution layer (16 filters, showing 8).

Inference: Different filters respond to different visual structures in the input image; some highlight the object's outline while others activate only on small localized regions, showing how early convolution filters extract distinct low-level features.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

```
Max Pooling output shape after first pool: (1, 16, 16, 32)
Average Pooling output shape after first pool: (1, 16, 16, 32)
Epoch 1/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 81s 56ms/step - accuracy: 0.4841 - loss: 1.4425 - val_accuracy: 0.5938 - val_loss: 1.1587
Epoch 2/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 80s 55ms/step - accuracy: 0.6263 - loss: 1.0703 - val_accuracy: 0.6578 - val_loss: 0.9896
Epoch 3/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 77s 54ms/step - accuracy: 0.6697 - loss: 0.9506 - val_accuracy: 0.6598 - val_loss: 0.9937
Epoch 4/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 83s 55ms/step - accuracy: 0.6990 - loss: 0.8674 - val_accuracy: 0.6766 - val_loss: 0.9441
Epoch 5/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 76s 54ms/step - accuracy: 0.7213 - loss: 0.7991 - val_accuracy: 0.6942 - val_loss: 0.8942
Epoch 1/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 69s 48ms/step - accuracy: 0.4359 - loss: 1.5602 - val_accuracy: 0.5274 - val_loss: 1.3120
Epoch 2/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 67s 48ms/step - accuracy: 0.5666 - loss: 1.2236 - val_accuracy: 0.5976 - val_loss: 1.1333
Epoch 3/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 83s 49ms/step - accuracy: 0.6186 - loss: 1.0827 - val_accuracy: 0.6200 - val_loss: 1.0786
Epoch 4/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 80s 47ms/step - accuracy: 0.6546 - loss: 0.9866 - val_accuracy: 0.6426 - val_loss: 1.0188
Epoch 5/5
1407/1407 ━━━━━━━━━━━━━━━━━━━━ 66s 47ms/step - accuracy: 0.6811 - loss: 0.9131 - val_accuracy: 0.6742 - val_loss: 0.9335
Max Pooling final val accuracy: 0.6941999793052673
Average Pooling final val accuracy: 0.6741999983787537
```

Figure 6: Pooling comparison.

Inference: Max pooling performed better, with a final validation accuracy of 69.42%, compared to 67.42% for average pooling. This shows that max pooling was more effective at retaining the most important features from the images, resulting in slightly better validation performance.

Task 6

Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

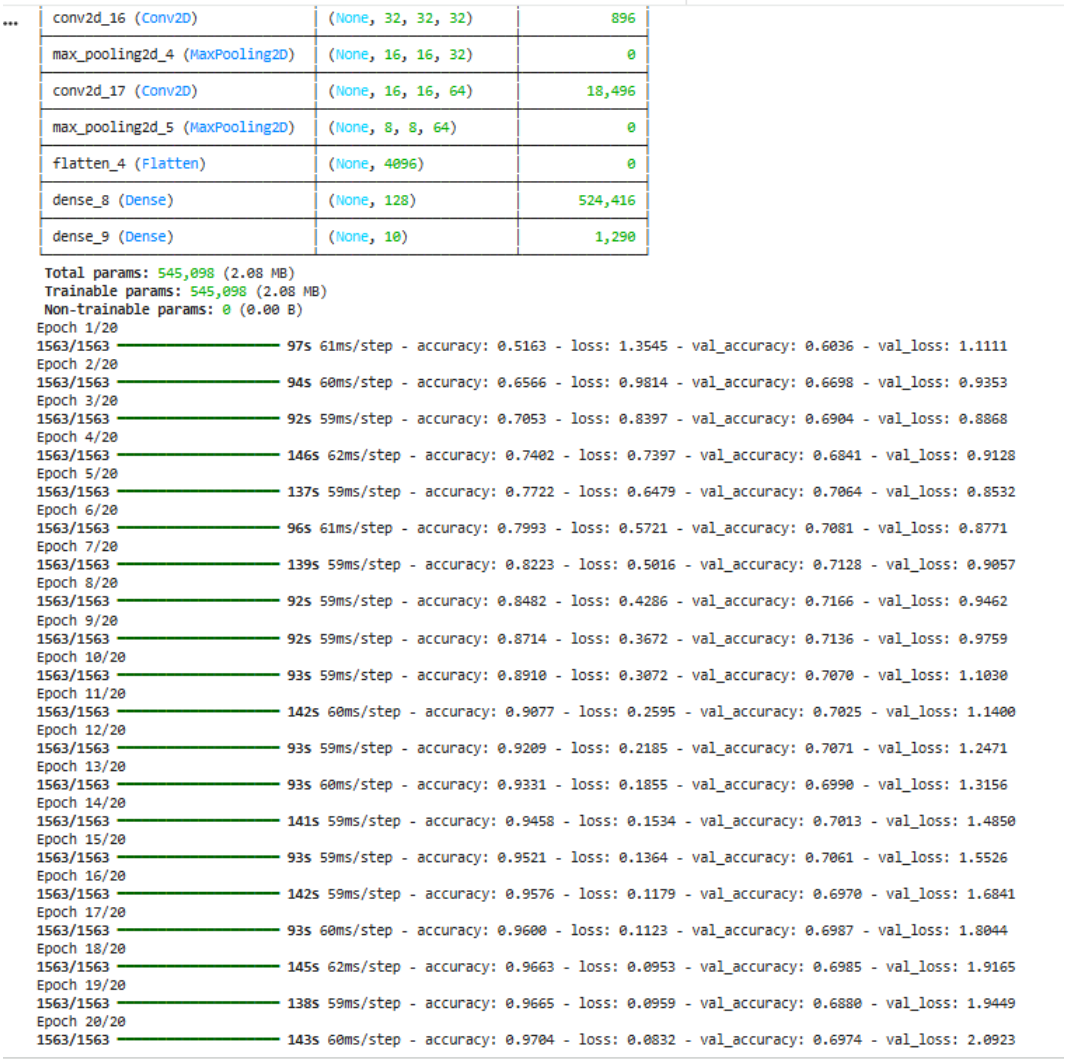


Figure 7: Training accuracy over epochs.

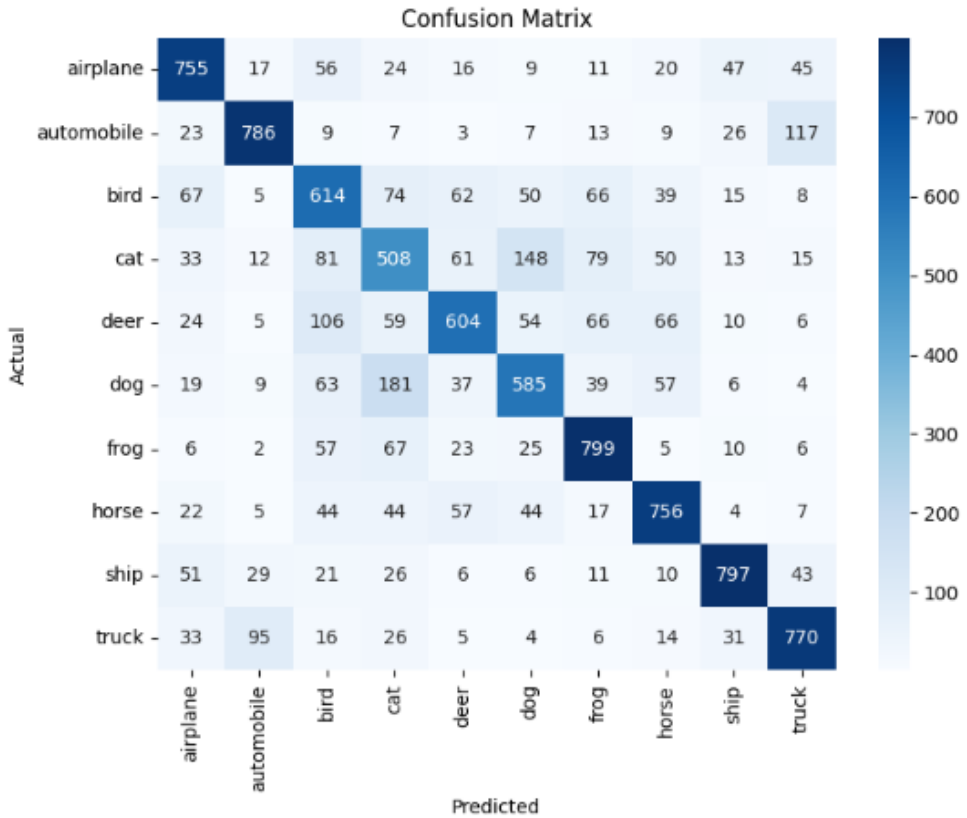
Inference: Training accuracy rises steadily across all 20 epochs, reaching 97.04% by the final epoch, showing the model fits the training data increasingly well.

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

```
Test Accuracy: 0.6974
Precision (macro): 0.6983441070499596
Recall (macro): 0.6974
F1-score (macro): 0.6971413097347817
```



	precision	recall	f1-score	support
airplane	0.73	0.76	0.74	1000
automobile	0.81	0.79	0.80	1000
bird	0.58	0.61	0.59	1000
cat	0.50	0.51	0.50	1000
deer	0.69	0.60	0.64	1000
dog	0.63	0.58	0.61	1000
frog	0.72	0.80	0.76	1000
horse	0.74	0.76	0.75	1000
ship	0.83	0.80	0.81	1000
truck	0.75	0.77	0.76	1000
accuracy			0.70	10000
macro avg	0.70	0.70	0.70	10000
weighted avg	0.70	0.70	0.70	10000

Figure 8: Confusion matrix of the evaluated model.

Inference: The model performs strongly on Automobile, Frog, Ship, and Horse. The most notable confusion occurs between Cat and Dog, and between Automobile and Truck – visually and semantically similar categories.

9. Mandatory Plots

Generate

1. Sample Images
2. Class Distribution

3. Training Accuracy
4. Validation Accuracy
5. Training Loss
6. Validation Loss
7. Feature Maps
8. Confusion Matrix

Write a 2–3 line inference for each plot.

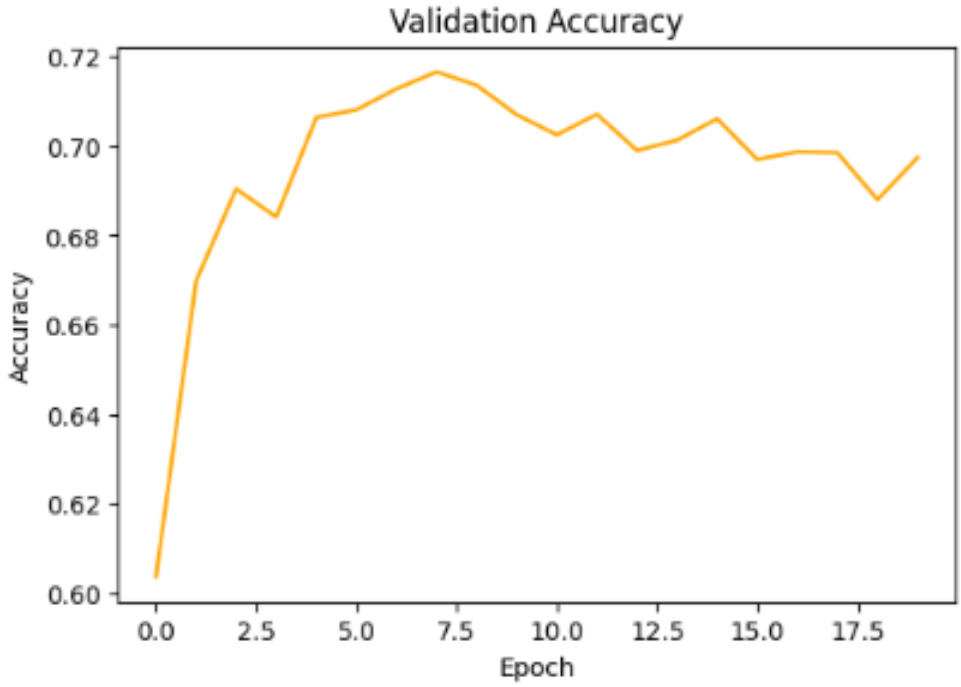


Figure 9: Validation accuracy over epochs.

Inference: Validation accuracy climbs quickly in the first few epochs and peaks around epoch 7–8 (near 71–72%), then plateaus and fluctuates slightly, indicating diminishing generalization gains beyond that point.

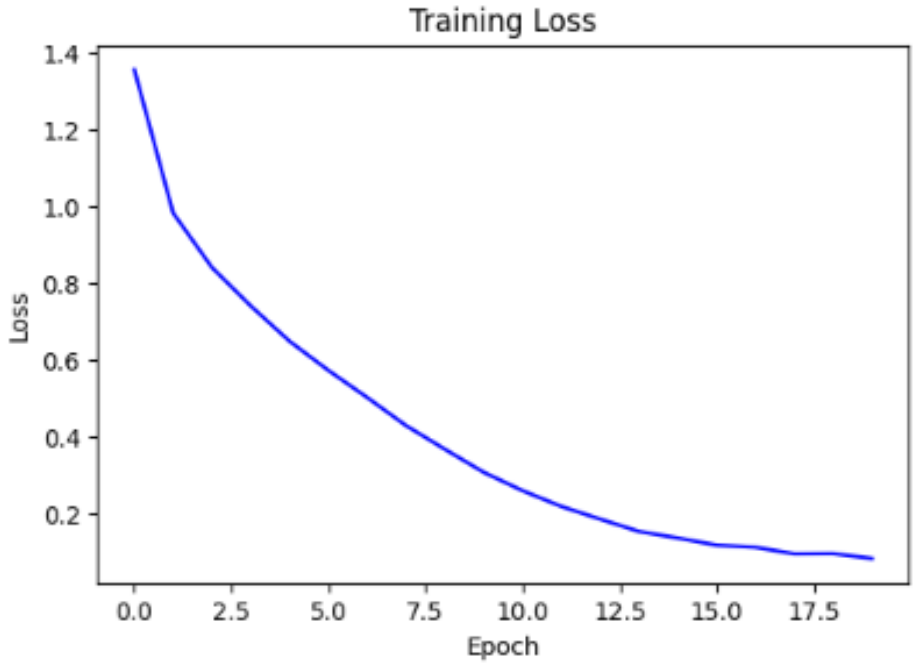


Figure 10: Training loss over epochs.

Inference: Training loss decreases consistently across all epochs, dropping to about 0.08 by the end, showing the model continues fitting the training set more closely.

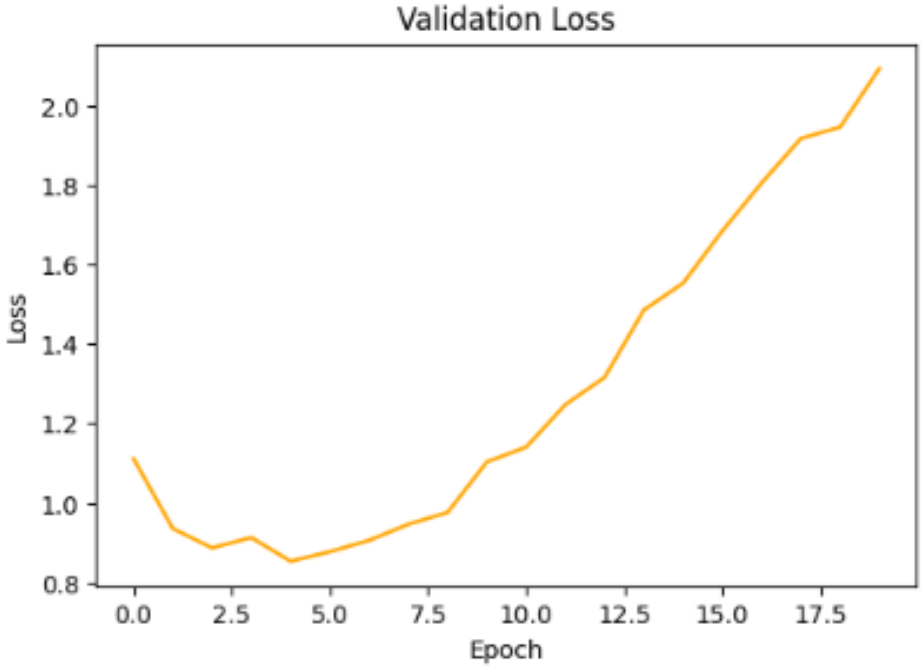


Figure 11: Validation loss over epochs.

Inference: Validation loss reaches its lowest point around epoch 4–5 and then rises steadily while training loss keeps falling – a clear sign of overfitting in later epochs.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

11. Results

- **Training Accuracy:** 97.04%
- **Testing Accuracy: 69.74%**
- **Precision (macro):** 0.6983
- **Recall (macro):** 0.6974
- **F1-score (macro):** 0.6971
- **Number of Parameters:** 545,098

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**
   Convolution preserves the spatial structure of images and detects local features like edges and textures using shared weights. It also uses far fewer parameters than fully connected layers for the same input size.
2. **How does stride affect the feature map size?**
   A larger stride moves the filter by more pixels, resulting in a smaller feature map. A smaller stride produces a larger, more finely-sampled feature map.
3. **What is the role of padding?**
   Padding adds extra pixels around the image to control the output size. It also helps preserve information from the edges of the image that would otherwise be under-represented.
4. **Why is pooling used?**
   Pooling reduces the spatial dimensions of feature maps, decreasing computation and the number of downstream parameters. It also helps the model focus on the most important features while adding some translation invariance.
5. **How do feature maps represent image characteristics?**
   Feature maps show the regions of the image that activate each convolution filter, such as edges, shapes, textures, and patterns. Different filters detect different characteristics of the same image.
6. **Why do CNNs require fewer parameters than MLPs?**
   CNNs use shared weights and local connections, so the same small filter is applied across the entire image instead of connecting every input pixel to every neuron.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.